

Music Taste Analysis

When I met my first girlfriend's friends for the first time, somebody innocently tried to make conversation with me by asking my music taste. I clammed up. How could I answer such a complicated question convincingly? There are so many things! I became annoyed with myself for not knowing what to say, not even any genre names. How uncool. My frustration leaked out, and my then-girlfriend later told me someone had asked what was wrong with me. (Hi, Sam.)

But *now* when someone asks, I have the perfect answer. "Would you like to see my data analysis!?" Enjoy lol.

How to Use

My first pass at this depended upon [Watsonbox's Exportify](#), but I decided I didn't like his version because of bugs and inadequate output detail. So I [went and forked it](#), cleaned up [the code](#), and [hosted it](#) myself.

As such, the code here depends on `.csv` inputs in the format output by [my version](#).

1. To get started, hop on over there, sign in to Spotify to give the app access to your playlists, and export whatever you like.
2. Next, either download this `.ipynb` file and run the notebook yourself or [launch it in Binder](#).
3. Either put the downloaded `.csv` in the same directory as the notebook, or upload it in Binder.
4. Open the `.ipynb` through your browser, update the `filename` variable in the first code cell to point to your playlist instead, and `shift+enter` in each following code cell to generate the corresponding plot. (Or select `Cell -> Run All` from the menu to make all graphs at once.)

Read the Data

For years I've been accumulating my favorite songs in a single master playlist called `music that tickles my fancy`. It's thousands of songs. This is what I'll be analyzing. Let's take a look at the first few rows to get a sense of what we're dealing with.

```
In [44]: filename = 'Liked_Songs.csv'

from matplotlib import pyplot
from matplotlib import font_manager
```

```

import seaborn
import pandas
from collections import defaultdict
from scipy.stats import pareto, gamma
from datetime import date

# Extra fonts for displaying other alphabets
candidate_fonts = [
    "Noto Sans CJK SC", # Linux CJK
    "Meiryo",           # Windows Japanese
    "Malgun Gothic",    # Windows Korean
    "Microsoft YaHei",  # Windows Simplified Chinese
    "PingFang SC",      # MacOS CJK
]
available_fonts = {f.name for f in font_manager.fontManager.ttflist} # Avail
usable_fonts = [f for f in candidate_fonts if (f in available_fonts)] # From
pyplot.rcParams["font.family"] = pyplot.rcParams["font.family"] + usable_fon

# read the data
data = pandas.read_csv(filename)
print("total songs:", data.shape[0])
print(data[:3])

```

total songs: 715

	Track URI	Track Name \
0	spotify:track:3T8Dmah1CqnFvW30C5QnVx	Get Dark
1	spotify:track:0RE6WvALkkwae00eB5yW30	Trash Bat
2	spotify:track:5pyVqlqThcxI7tPDjHIzPh	Climax

	Album Name	Artist Name(s)	Release Date \
0	The Missing Man	AFI	2018-12-07
1	The Missing Man	AFI	2018-12-07
2	Looking 4 Myself (Expanded Edition)	USHER	2012-06-12

	Duration (ms)	Popularity	Explicit	Added By	Added At	...
0	160923	16	False	NaN	2025-09-18T18:29:13Z	...
1	127625	17	False	NaN	2025-09-18T18:29:09Z	...
2	232905	62	False	NaN	2025-09-02T06:21:42Z	...

	Key	Loudness	Mode	Speechiness	Acousticness	Instrumentalness	Liveness
0	9.0	-1.789	0.0	0.0445	0.000296	0.000117	0.0965
1	0.0	-1.541	1.0	0.0644	0.001280	0.046000	0.1130
2	8.0	-8.837	1.0	0.0782	0.547000	0.000027	0.1560

	Valence	Tempo	Time Signature
0	0.680	102.005	4.0
1	0.758	102.491	4.0
2	0.333	135.877	4.0

[3 rows x 24 columns]

Artist Bar Chart

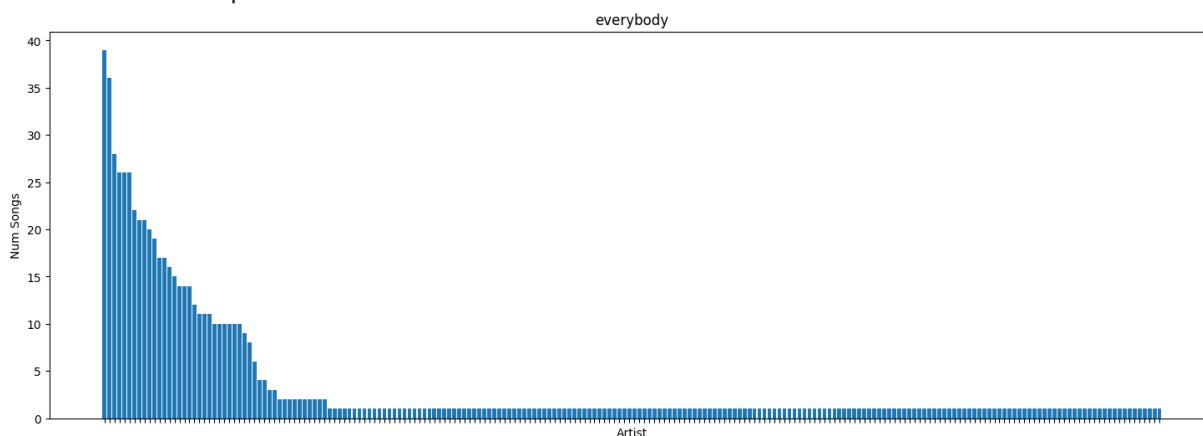
Number of songs binned by artist.

```
In [45]: # count songs per artist
artists = defaultdict(int)
for i, song in data.iterrows():
    if isinstance(song['Artist Name(s)'], str):
        for musician in song['Artist Name(s)'].split(','):
            artists[musician] += 1

# sort for chart
artists = pandas.DataFrame(artists.items(), columns=['Artist', 'Num Songs']
                           ).sort_values('Num Songs', ascending=False).reset_
print("number of unique artists:", artists.shape[0])

pyplot.figure(figsize=(18, 6))
pyplot.bar(artists['Artist'], artists['Num Songs'])
pyplot.xticks(visible=False)
pyplot.xlabel(artists.columns[0])
pyplot.ylabel(artists.columns[1])
pyplot.title('everybody')
pyplot.show()
```

number of unique artists: 212



Note I've attributed songs with multiple artists to multiple bars, so the integral here is the number of unique song-artist pairs, not just the number of songs.

It seems to follow a Pareto distribution. Let's try to fit one.

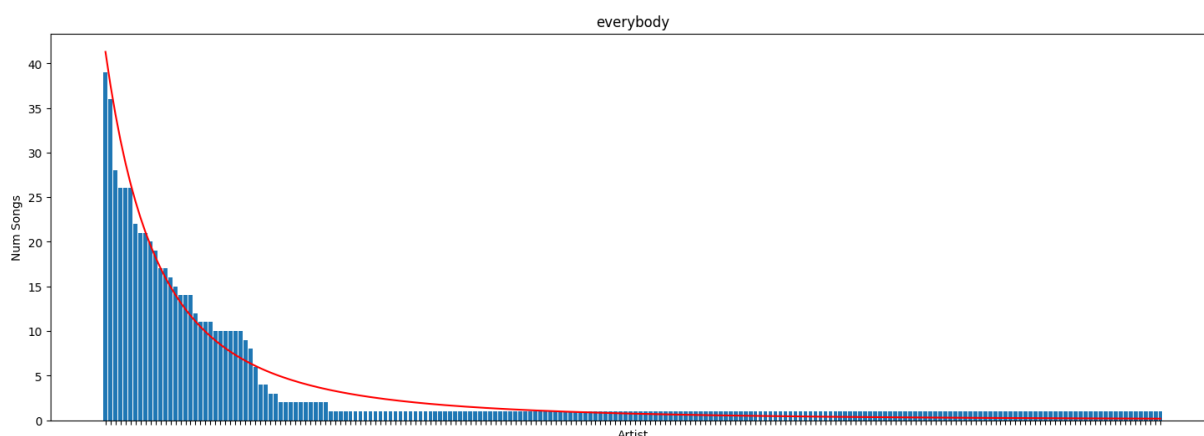
```
In [46]: # Let's find the best parameters. Need x, y data 'sampled' from the distribu
# parameter fit.
y = []
for i in range(artists.shape[0]):
    for j in range(artists['Num Songs'][i]):
        y.append(i) # just let y have index[artist] repeated for each

# sanity check. If the dataframe isn't sorted properly, y isn't either.
#pyplot.figure()
#pyplot.hist(y, bins=30)

# The documentation is pretty bad, but this is okay:
# https://stackoverflow.com/questions/6620471/fitting-empirical-distribution
```

```
# ones-with-scipy-python
param = pareto.fit(y, 100)
pareto_fitted = len(y)*pareto.pdf(range(artists.shape[0]), *param)
# param = gamma.fit(y) # gamma fits abysmally; see for yourself by uncomment
# gamma_fitted = len(y)*gamma.pdf(range(artists.shape[0]), *param)

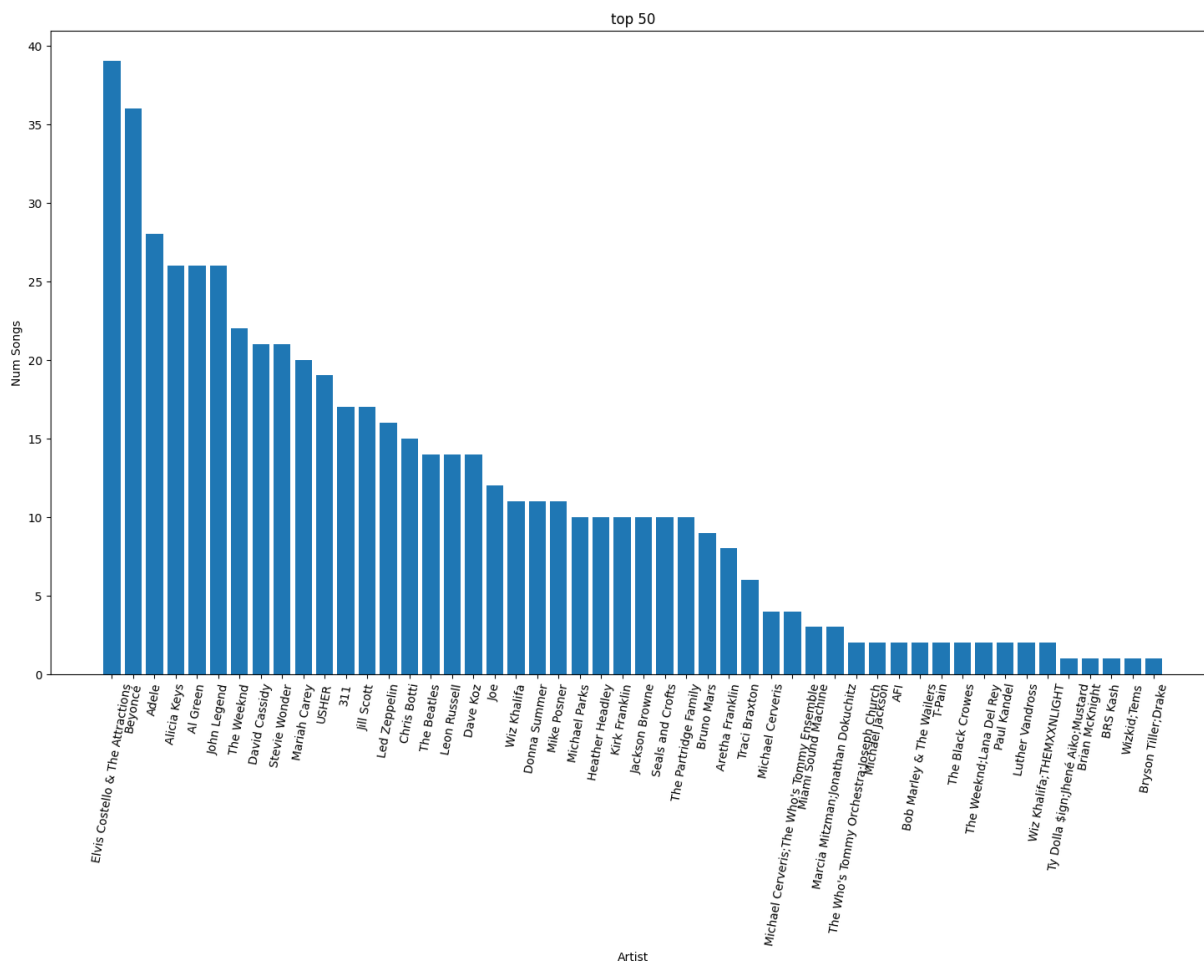
pyplot.figure(figsize=(18, 6))
pyplot.bar(artists['Artist'], artists['Num Songs'])
pyplot.plot(pareto_fitted, color='r')
#pyplot.plot(gamma_fitted, color='g')
pyplot.xticks(visible=False)
pyplot.xlabel(artists.columns[0])
pyplot.ylabel(artists.columns[1])
pyplot.title('everybody');
```



Best fit is still too sharp for the data, and I tried for a good long while to get it to fit better, so I conclude this doesn't *quite* fit a power law.

Let's plot the top 50 artists so we can actually read who they are.

```
In [47]: pyplot.figure(figsize=(18, 10))
pyplot.bar(artists['Artist'][:50], artists['Num Songs'][:50])
pyplot.xticks(rotation=80)
pyplot.xlabel(artists.columns[0])
pyplot.ylabel(artists.columns[1])
pyplot.title('top 50');
```

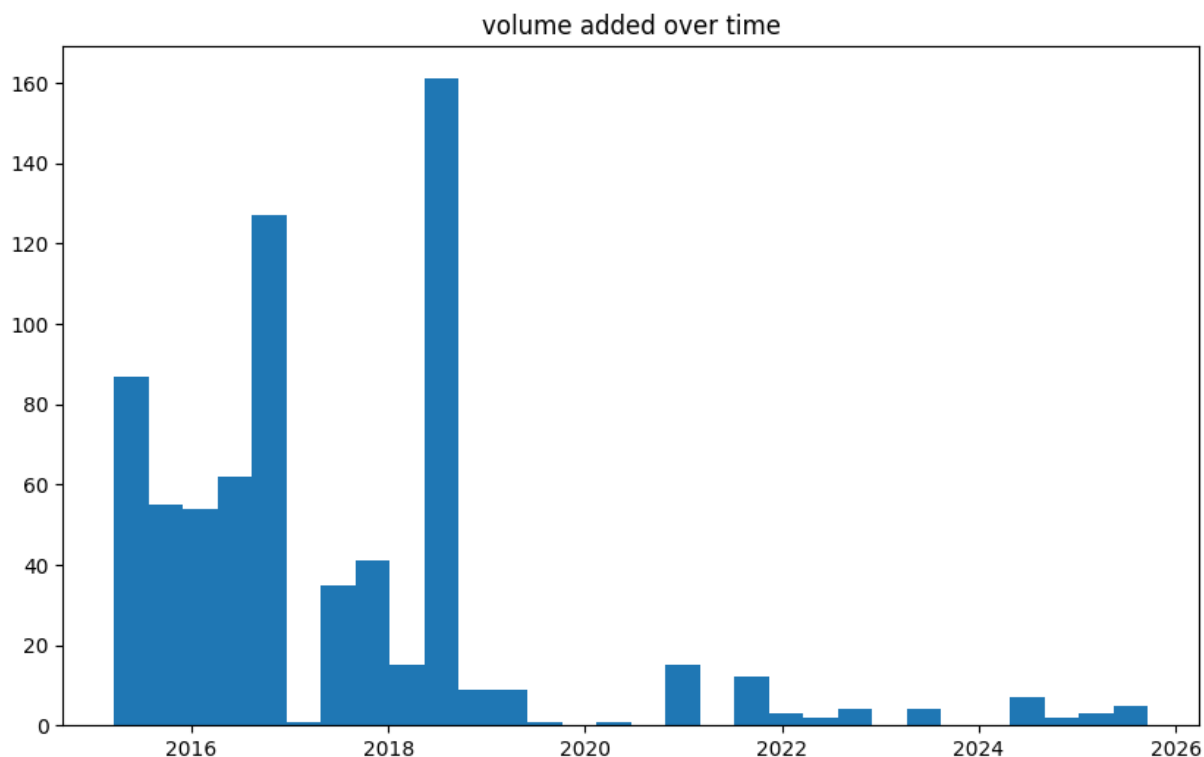


Volume Added Over Time

My proclivity to add songs to this playlist is a proxy for my interest in listening to music generally. How has it waxed and waned over time?

```
In [48]: from pandas.plotting import register_matplotlib_converters
register_matplotlib_converters() # to suppress warning

# Plot of added volume over time
parse_date = lambda d:(int(d[:4]), int(d[5:7]), int(d[8:10]))
pyplot.figure(figsize=(10, 6))
pyplot.hist([date(*parse_date(d)) for d in data['Added At']], bins=30)
pyplot.title('volume added over time');
```



The initial spike is from when I first started using Spotify as the home for this collection and manually added hundreds from my previous list. There's also a funny Covid bump from when I was woodworking and listening a lot.

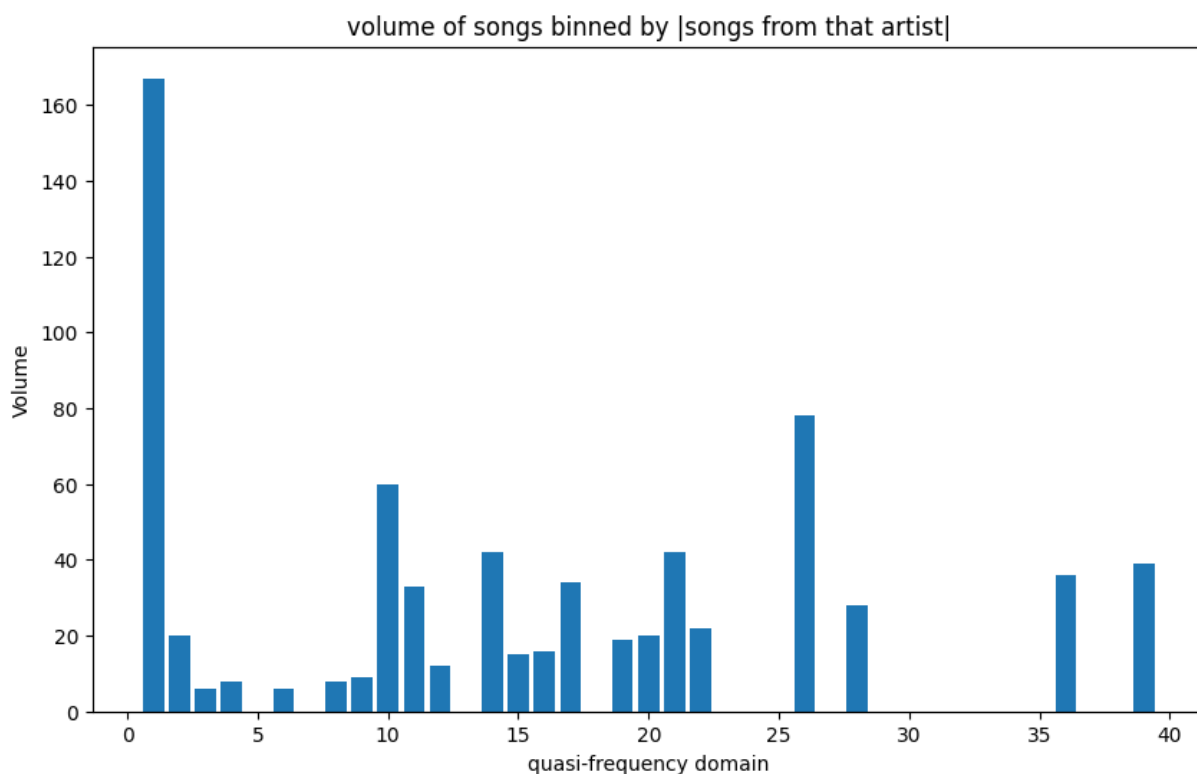
Eclecticism Measure (Frequency Transform)

This one is a personal favorite. I want to know how many of my songs are one-offs from that artist for me--just individual pieces I found fantastic and ended up adding after a few listens--, how many are two-offs, et cetera. I know it must be heavily skewed toward the low numbers.

```
In [49]: # bar chart of first bar chart == hipster diversity factor
frequency = defaultdict(int)
for n in artists['Num Songs']:
    frequency[n] += n
frequency = pandas.DataFrame(frequency.items(), columns=['Unique Count', 'Volume'])
frequency.sort_values('Volume', ascending=False)
print("number of song-artist pairs represented in the eclecticism chart:",
      sum(frequency['Volume']))

pyplot.figure(figsize=(10, 6))
pyplot.bar(frequency['Unique Count'].values, frequency['Volume'].values)
pyplot.title('volume of songs binned by |songs from that artist|')
pyplot.xlabel('quasi-frequency domain')
pyplot.ylabel(frequency.columns[1]);
```

number of song-artist pairs represented in the eclecticism chart: 720



So, yes, it's much more common for an artist to make it in my list a few times than many times. In fact, the plurality of my top songs come from unique artists.

Conversely, this view also makes stark those few musicians from whom I've collected dozens.

Note that here, as in the artist bar charts, some songs are doubly-counted, because in cases artists collaborated I listed the song in both bins.

Genres Bar Chart

Alright, enough messing around. All the above were possible with the output from Watsonbox's Exportify. Let's get to the novel stuff you came *here* for.

People describe music by genre. As we'll see, genre names are flippin' hilarious and extremely varied, but in theory if I cluster around a few, that should give you a flavor of my tastes.

```
In [50]: # count songs per genre
genres = defaultdict(int)
for i, song in data.iterrows():
    if type(song['Genres']) is str: # some times there aren't any, and this
        for genre in song['Genres'].split(','):
            if len(genre) > 0: # empty string seems to be a legit genre
                genres[genre] += 1

# sort for chart
genres = pandas.DataFrame(genres.items(), columns=['Genre', 'Num Songs'])
```

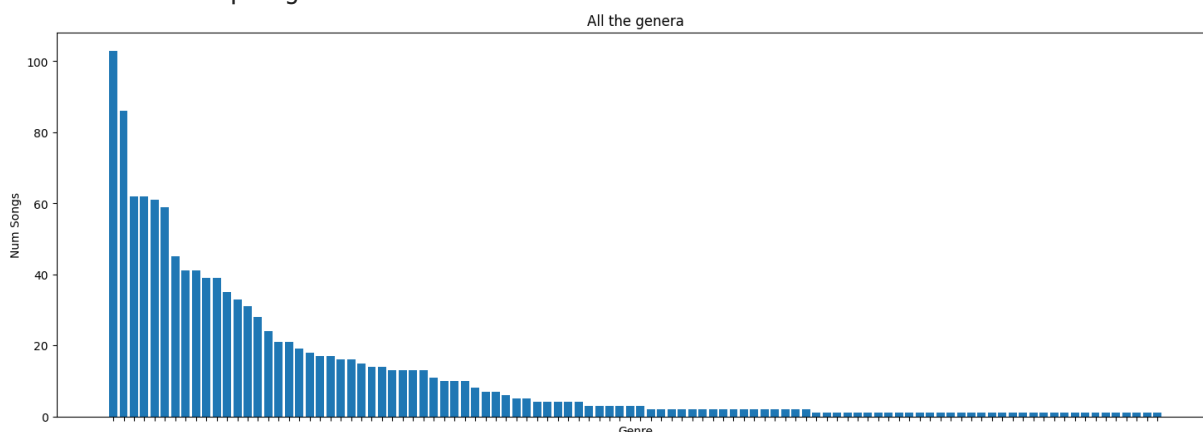
```

        ).sort_values('Num Songs', ascending=False).reset_
print("number of unique genres:", genres.shape[0])

pyplot.figure(figsize=(18, 6))
pyplot.bar(genres['Genre'], genres['Num Songs'])
pyplot.xticks(visible=False)
pyplot.xlabel(genres.columns[0])
pyplot.ylabel(genres.columns[1])
pyplot.title('All the genera');

```

number of unique genres: 102



So many! Let's do the same thing as with the artists and for giggles see if it fits a power law.

```

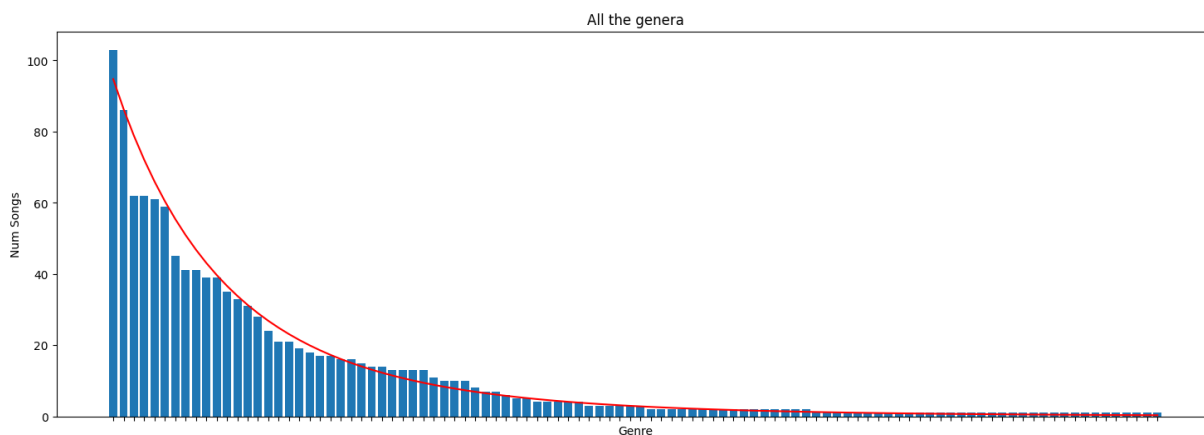
In [51]: y = []
for i in range(genres.shape[0]):
    for j in range(genres['Num Songs'][i]):
        y.append(i)

# sanity check
#pyplot.figure()
#pyplot.hist(y, bins=30)

param = pareto.fit(y, 100)
pareto_fitted = len(y)*pareto.pdf(range(genres.shape[0]), *param)

pyplot.figure(figsize=(18, 6))
pyplot.bar(genres['Genre'], genres['Num Songs'])
pyplot.plot(pareto_fitted, color='r')
pyplot.xticks(visible=False)
pyplot.xlabel(genres.columns[0])
pyplot.ylabel(genres.columns[1])
pyplot.title('All the genera');

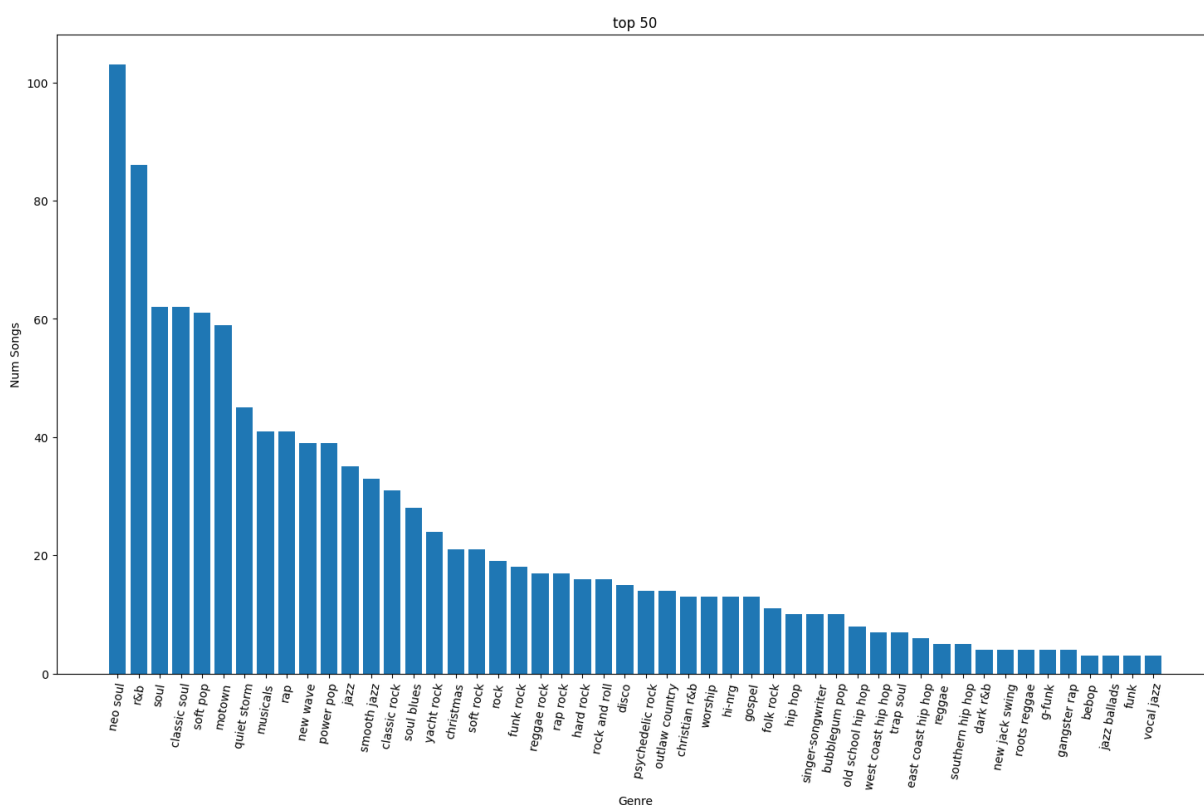
```



Still too sharp, but fits better than with the artists.

Let's look at the top 50 so we can read the names.

```
In [52]: pyplot.figure(figsize=(18, 10))
pyplot.bar(genres['Genre'][:50], genres['Num Songs'][:50])
pyplot.xticks(rotation=80)
pyplot.xlabel(genres.columns[0])
pyplot.ylabel(genres.columns[1])
pyplot.title('top 50');
```

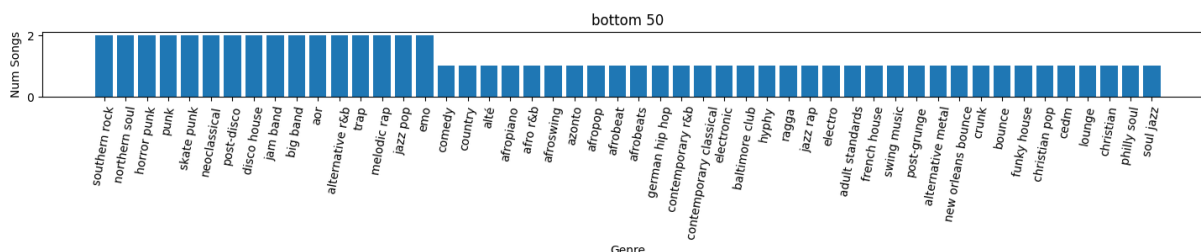


"Indie popitism" lol. wtf? "Dreamo", "Vapor soul", "Freak folk", "Tropical house", "Post-grunge", "Hopebeat", "Noise pop", "Mellow gold"

These are too good. Next time someone asks me my music taste, I'm definitely using these.

If these are the *most* popular names, what are the really unique ones at the bottom of the chart?

```
In [53]: pyplot.figure(figsize=(18, 1))
pyplot.bar(genres['Genre'][-50:], genres['Num Songs'][-50:])
pyplot.xticks(rotation=80)
pyplot.xlabel(genres.columns[0])
pyplot.ylabel(genres.columns[1])
pyplot.title('bottom 50');
```



"hauntology", "psychadelic folk", "stomp and whittle", "dark trap", "filthstep",
 "shamanic", "deep underground hip hop", "future garage", "crunk", "sexy drill", "hard
 house"

That was fun.

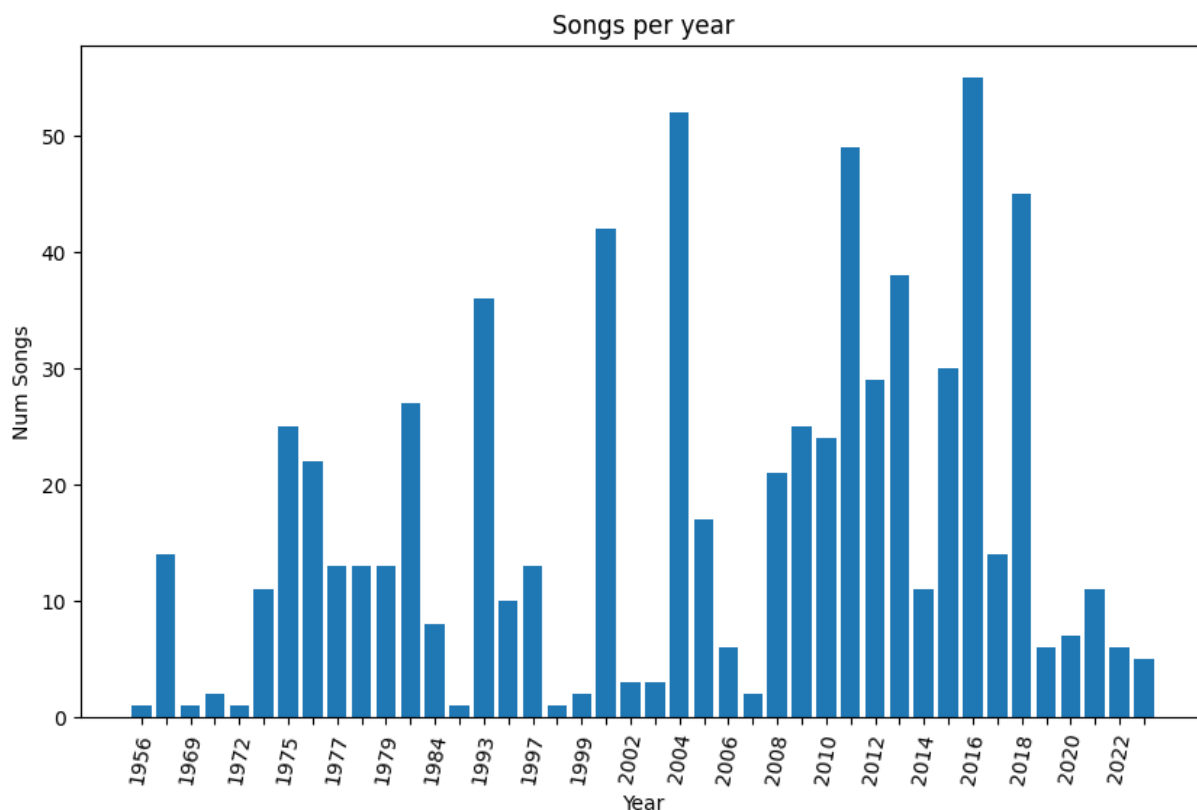
Release Dates

Which era of music do I prefer?

```
In [54]: years = defaultdict(int)
for i, song in data.iterrows():
    if isinstance(song['Release Date'], str): # somebody found a NaN release
        years[song['Release Date'][:4]] += 1

years = pandas.DataFrame(years.items(), columns=['Year', 'Num Songs'])
    .sort_values('Year')

pyplot.figure(figsize=(10, 6))
pyplot.bar(years['Year'], years['Num Songs'])
pyplot.xticks(years['Year'], [y if i % 2 == 0 else '' for i, y in enumerate(y
pyplot.xlabel(years.columns[0])
pyplot.ylabel(years.columns[1])
pyplot.title('Songs per year');
```



It seems to follow a Gamma distribution! This makes sense because I'm more likely to have heard things that are nearer me in time, and it takes a while for them to get through my process and become favorites.

Let's fit that gamma to the time-reversed data.

```
In [55]: # Some years are missing, so transform to a dataframe that covers full time
eldest = int(years['Year'].values[0])
youngest = int(years['Year'].values[-1])
missing_years = [str(x) for x in range(eldest+1, youngest) if
                  str(x) not in years['Year'].values]
ago = pandas.concat((years, pandas.DataFrame.from_dict(
    {'Year': missing_years, 'Num Songs': [0 for x in range(len(missing_years))]}),
    sort_values('Year', ascending=False).reset_index(drop=True))

y = []
for i in range(ago.shape[0]):
    for j in range(int(ago['Num Songs'][i])):
        y.append(i)

# sanity check histogram to make sure I'm constructing y properly
#pyplot.figure()
#pyplot.hist(y, bins=30)

param = gamma.fit(y, 10000)
gamma_fitted = len(y)*gamma.pdf(range(ago.shape[0]), *param)

pyplot.figure(figsize=(10, 6))
pyplot.bar(range(len(ago['Year'])), ago['Num Songs'])
```

```

pyplot.plot(gamma_fitted, color='g')
pyplot.xlabel('Years Ago')
pyplot.ylabel(ago.columns[1])
pyplot.title('Songs per year (in absolute time)');

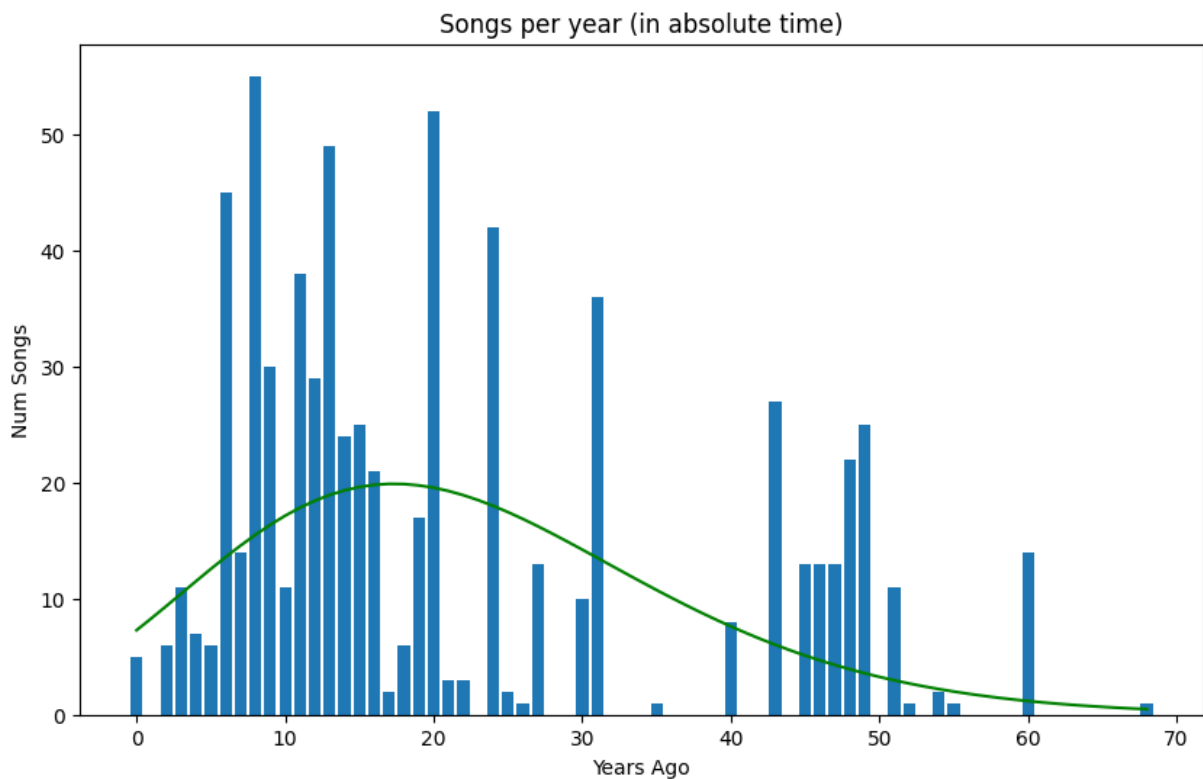
print('Oldest Hall of Fame')
print(data[['Track Name', 'Artist Name(s)', 'Release Date']].sort_values(
    'Release Date')[:10])

```

Oldest Hall of Fame

	Track Name \
328	The Nearness Of You
357	I'm A Loser – Remastered 2009
356	No Reply – Remastered 2009
358	Baby's In Black – Remastered 2009
359	Rock And Roll Music – Remastered 2009
360	I'll Follow The Sun – Remastered 2009
361	Mr Moonlight – Remastered 2009
362	Kansas City / Hey-Hey-Hey-Hey – Medley / Remas...
364	Words Of Love – Remastered 2009
365	Honey Don't – Remastered 2009

	Artist Name(s)	Release Date
328	Ella Fitzgerald;Louis Armstrong	1956-01-01
357	The Beatles	1964-12-04
356	The Beatles	1964-12-04
358	The Beatles	1964-12-04
359	The Beatles	1964-12-04
360	The Beatles	1964-12-04
361	The Beatles	1964-12-04
362	The Beatles	1964-12-04
364	The Beatles	1964-12-04
365	The Beatles	1964-12-04



Pretty good fit! I seem to be extra partial to music from about 5 years ago. We'll see whether the present or maybe the further past catches up.

Popularity Contest

I was happy to find popularity listed as a field in Spotify's track JSON. It's a percentile between 0 and 100, rather than an absolute number of plays. Still, it can be used to give a notion of how hipster I am.

```
In [56]: popularity = defaultdict(int)
for i, song in data.iterrows():
    popularity[song['Popularity']] += 1

popularity = pandas.DataFrame(popularity.items(), columns=['Popularity', 'Num
                ]).sort_values('Popularity')

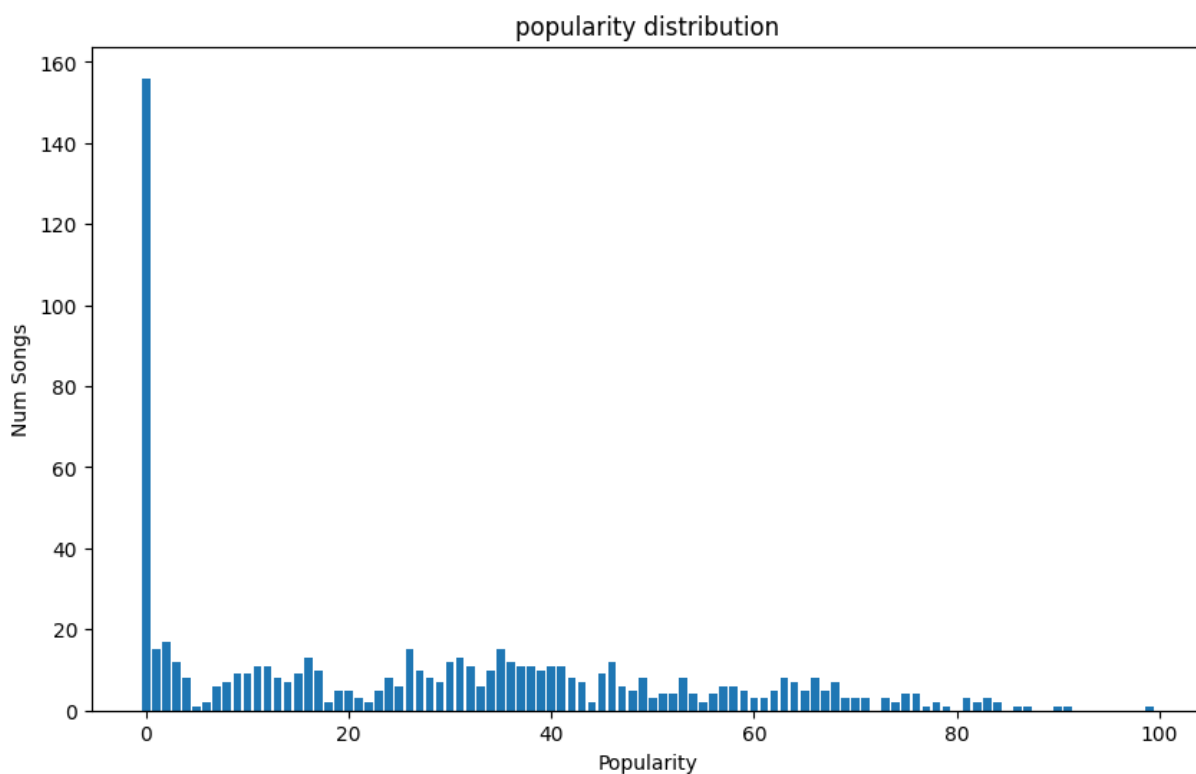
pyplot.figure(figsize=(10, 6))
pyplot.bar(popularity['Popularity'].values, popularity['Num Songs'].values)
pyplot.xlabel(popularity.columns[0])
pyplot.ylabel(popularity.columns[1])
pyplot.title('popularity distribution');

print("Average song popularity: ", popularity['Popularity'].mean())
print("Median song popularity: ", popularity['Popularity'].median())
print("Max song popularity: ", popularity['Popularity'].max())
```

Average song popularity: 43.98863636363637

Median song popularity: 43.5

Max song popularity: 99



Damn, I'm a hipster.

Track Duration

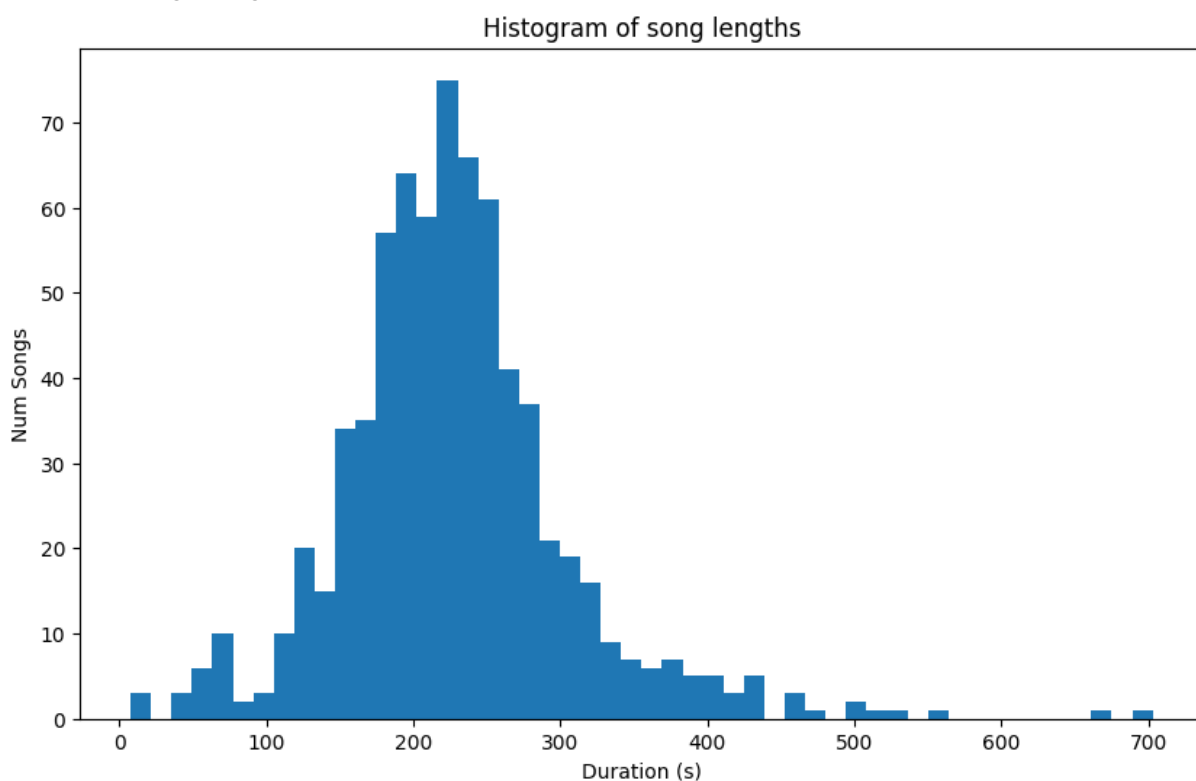
Do I prefer long songs or short ones?

```
In [57]: pyplot.figure(figsize=(10,6))
pyplot.hist(data['Duration (ms)']/1000, bins=50);
pyplot.xlabel('Duration (s)')
pyplot.ylabel('Num Songs')
pyplot.title('Histogram of song lengths')

mean = data['Duration (ms)'].mean()/1000
median = data['Duration (ms)'].median()/1000
print("Average song length: " + str(int(mean//60)) + (":" if mean%60 >=10 else ""
      + str(mean%60))
print("Median song length: " + str(int(median//60)) + (":" if median%60 >=10 else ""
      + str(median%60))
```

Average song length: 3:48.33195524475525

Median song length: 3:43.985999999999999



Median is lower than the mean, so I'm skewed right. That is, I like a few really long songs. What are they?

```
In [58]: print("Longest Hall of Fame:")
print(data[['Track Name', 'Artist Name(s)', 'Release Date', 'Duration (ms)']
      ['Duration (ms)', ascending=False][:10])
```

Longest Hall of Fame:

	Track Name \
249	No More Tears (Enough Is Enough) – 12" Version
225	In My Time of Dying – 1990 Remaster
78	Crazy Baldhead / Running Away – Live
229	In the Light – 1990 Remaster
680	Black Man
684	Another Star
228	Kashmir – 1990 Remaster
68	May I (feat. Mr. Talkbox)
156	Psilocybin (Love In Full Effect)
625	Just Right Tonight

	Artist Name(s)	Release Date	Duration (ms)
249	Donna Summer;Barbra Streisand	1979-10-15	703133
225	Led Zeppelin	1975-02-24	665600
78	Bob Marley & The Wailers	1977-01-01	561597
229	Led Zeppelin	1975-02-24	526586
680	Stevie Wonder	1976-09-28	509506
684	Stevie Wonder	1976-09-28	508226
228	Led Zeppelin	1975-02-24	508200
68	T-Pain;Mr. Talkbox	2017-11-17	473765
156	Jhené Aiko;Dr. Chill	2017-09-22	466280
625	Aretha Franklin	1984-11-14	465426

Musical Features

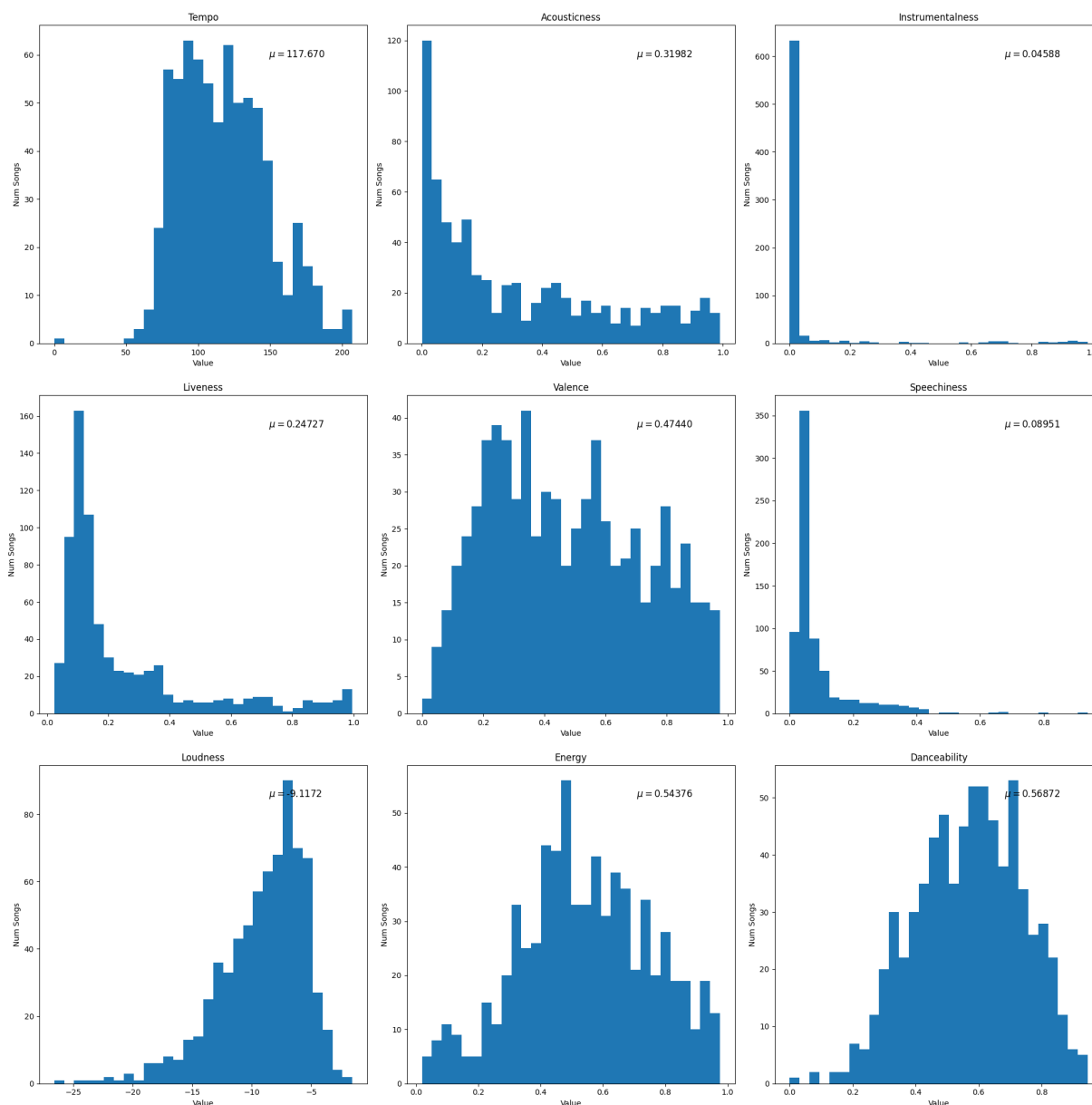
In the interest of understanding user tastes and providing the best possible music recommendations, Spotify has done [some really sophisticated analysis](#) of actual track content, which has only gotten more extensive in recent years. Music is a time series, but most similarity metrics (and most ML methods generally) require inputs to be vectors, that is: points in some feature-space. So they've transformed the tracks to numerical metrics like Energy and Valence (continuous) and Key (discrete).

For the continuous metrics, here are distributions for my songs.

```
In [59]: pyplot.figure(figsize=(20,20))

for i,category in enumerate(['Tempo', 'Acousticness', 'Instrumentalness', 'L
                             'Valence', 'Speechiness', 'Loudness', 'Energy',
    pyplot.subplot(3, 3, i+1)
    pyplot.hist(data[category], bins=30)
    pyplot.text(pyplot.xlim()[1] - (pyplot.xlim()[1] - pyplot.xlim()[0])*0.3
                pyplot.ylim()[1]*0.9, r'$\mu$'+str(data[category].mean())[0],
    pyplot.xlabel('Value')
    pyplot.ylabel('Num Songs')
    pyplot.title(category)

pyplot.tight_layout(h_pad=2)
```



My Valence is somewhat negatively skewed; do I have an affinity for sadder songs?

Now let's look at the discrete music features.

```
In [60]: pyplot.figure(figsize=(15,4))

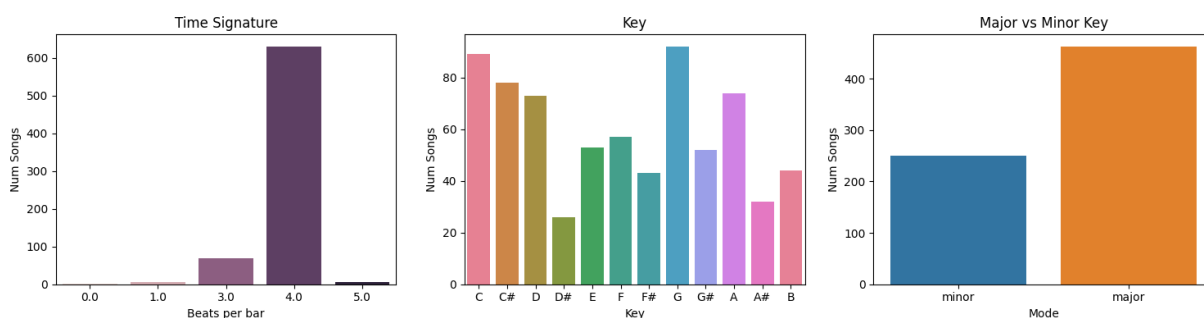
pyplot.subplot(1, 3, 1)
seaborn.countplot(data, x='Time Signature', hue='Time Signature', legend=False)
pyplot.xlabel('Beats per bar')
pyplot.ylabel('Num Songs')
pyplot.title('Time Signature')

pyplot.subplot(1, 3, 2)
seaborn.countplot(data, x='Key', hue='Key', palette='husl', legend=False)
pyplot.xticks(ticks=pyplot.xticks()[0], labels=['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B'])
pyplot.ylabel('Num Songs')
pyplot.title('Key')

pyplot.subplot(1, 3, 3)
```

```
seaborn.countplot(data, x='Mode', hue='Mode', legend=False)
pyplot.xticks(ticks=pyplot.xticks()[0], labels=['minor', 'major'])
pyplot.ylabel('Num Songs')
pyplot.title('Major vs Minor Key');

pyplot.tight_layout(w_pad=2)
```



Musicians seem to favor C major and eschew D#. More than a third of my songs are in a minor key. I don't have a baseline to compare against here, but this might contribute to my lower Valence.

Looks like the vast majority of my music is 4/4 time with a good few in 3/4. I wasn't even aware there were any with 5 beats. What are those?

```
In [61]: print('5:\n', data.loc[data['Time Signature']==5][
          ['Track Name', 'Artist Name(s)', 'Release Date'][:20])
```

```
5:
      Track Name Artist Name(s) \
117 How Come You Don't Call Me - Unplugged Live at... Alicia Keys
295                               I Know Better John Legend
331                Have Yourself A Merry Little Christmas Dave Koz
335 The Christmas Song (Chestnuts Roasting On An O... Dave Koz
361                Mr Moonlight - Remastered 2009 The Beatles
503    Melt My Heart to Stone - Live At The Hotel Cafe Adele

      Release Date
117    2005-10-11
295    2016-12-02
331    2011-01-01
335    2011-01-01
361    1964-12-04
503    2008-01-28
```

Make A Fist is totally 5/4, and so is Animals. Funny how I didn't notice the strange energetic time signature until now. But Carry On is definitely 4/4, as is Yachts, and Pray is 6/8. So Spotify's algorithm isn't perfect at this, which is expected.

What are 0 and 1?

```
In [62]: print('0:\n', data.loc[data['Time Signature']==0][
        ['Track Name', 'Artist Name(s)', 'Release Date'][:10])
print('\n1:\n', data.loc[data['Time Signature']==1][
        ['Track Name', 'Artist Name(s)', 'Release Date'][:20])
```

0:

	Track Name	Artist Name(s)	Release Date
8	Souvenir d'Italie	Lelio Luttazzi	2009-01-01

1:

	Track Name	\
19	Come Through and Chill (feat. J. Cole & Salaam...	
59	Been A While	
112	Intro Alicia's Prayer (Acappella) – Unplugged ...	
166	How Love Should Be	
498	Make You Feel My Love	
594	De Novo Adagio (Intro)	

	Artist Name(s)	Release Date
19	Miguel;J. Cole;Salaam Remi	2017-12-01
59	6LACK;Love Renaissance (LVRN)	2018-12-21
112	Alicia Keys	2005-10-11
166	Chris Botti	2004-09-28
498	Adele	2008-01-28
594	Alicia Keys	2012-11-26

Looks like there is only one song with 0 time signature. It's a piano piece with a tempo that rises and falls. This category might be for variable tempo, or unknown.

Claire De Lune is 9/8 time, so sort of waltzish but not really.

The Major General's Song is 4/4, but there are some stops in there and a lot of speaking, so I understand how that might be difficult to pick out. Same with Campfire Song Song lol.

Top of the Hill really sounds like 7/4 to me (1-2-123 sort of beat).

Pú ert jörðin is actually properly 1/4 time according to the internet, and relistening I understand how that could be the case. It's like there are little riffs each bar following a quadruplet pattern, but the major beats really only come every bar.

The Last of Us (You and Me) seems similar. It might be properly 1/4 time.

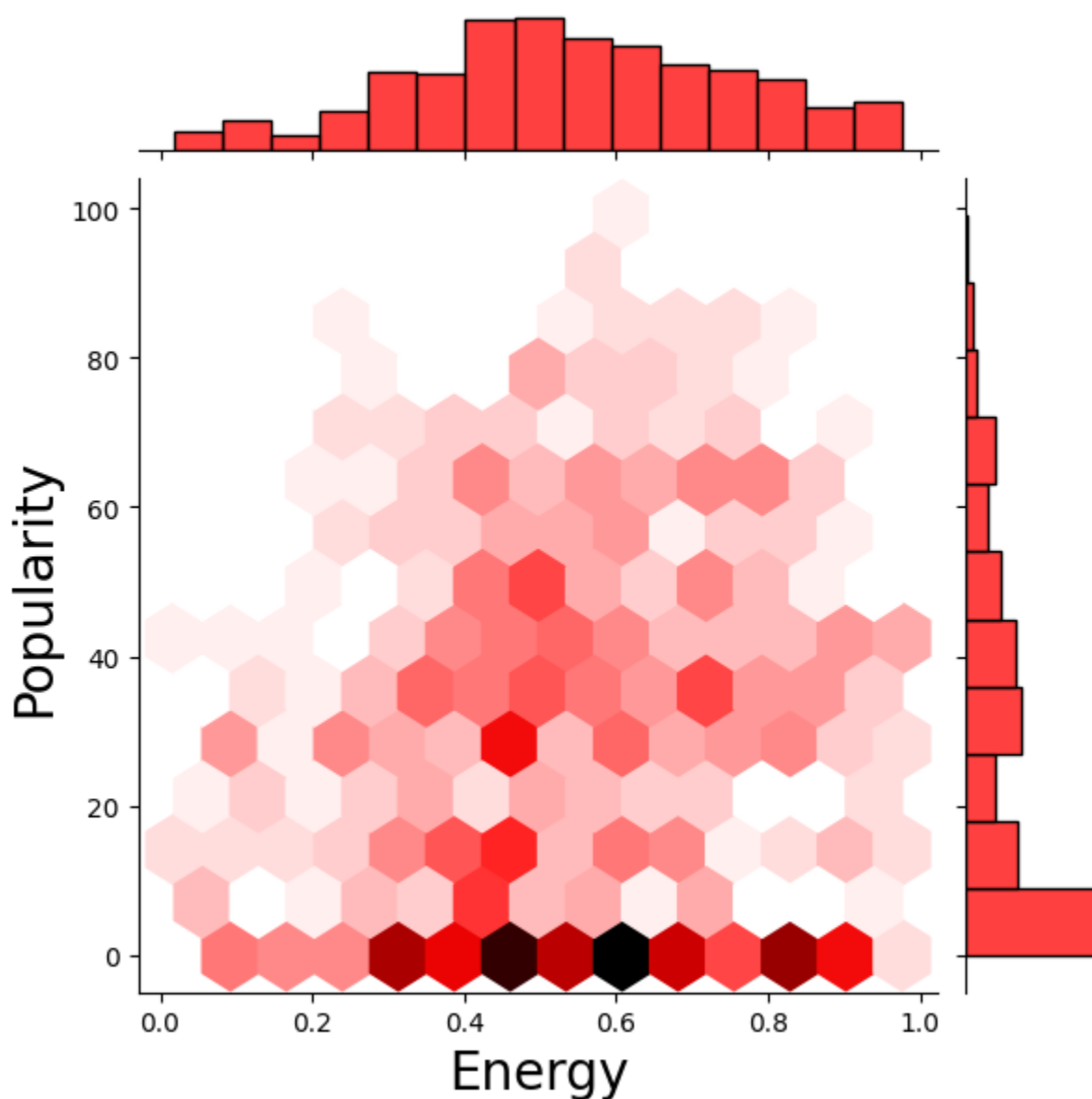
So it looks like this category is for actual single beats and unusual time signatures that Spotify isn't sure what to do with.

Joint Analysis

I mostly just want to showcase what's possible. Let's plot Energy and Popularity together to see whether there is a relationship.

```
In [63]: x = 'Energy'
y = 'Popularity'

axes = seaborn.jointplot(x=data[x], y=data[y], kind='hex', color='r')
axes.set_axis_labels(x, y, fontsize=20);
```



The density is pretty scattered, doesn't the whole plot, meaning the relationship here is actually pretty weak. Surprising.

The Final Frontier

Finally, I'm going to follow [this guy's example](#) and feed the dimension-reduced data to a one-class SVM to get a sense of what the frontier of my normal taste looks like in that space, [heat-map-of-the-universe-style](#).

[t-SNE](#) is a method for visualizing high-dimensional data in low-dimension. Songs which

are more alike will be nearer each other in the feature space, but we can't visualize a space with that many dimensions. What we can do is reconstitute the points in 2D, attempting to preserve the pairwise distances, the notions of similarity, between songs.

```
In [64]: show_percent = 2

from sklearn.manifold import TSNE
from random import random
from sklearn.svm import OneClassSVM
import numpy

# Create a dataframe of only the numerical features, all normalized so embed
# doesn't get confused by scale differences
numerical_data = data.drop(['Track URI', 'Track Name', 'Album Name', 'Artist',
                            'Explicit', 'Added By', 'Added At', 'Genres', 'Record Label'])
numerical_data['Release Date'] = pandas.to_numeric(
    numerical_data['Release Date'].str.slice(0,4))
numerical_data = (numerical_data - numerical_data.mean())/numerical_data.std
print('using:', list(numerical_data.columns))

# If you like, only include a subset of these, because the results with all
# is really hard to interpret
#tsne_data = numerical_data[['Popularity', 'Energy', 'Acousticness',
#                             'Valence', 'Loudness']]
#
#print("\nConsidering similarity with respect to the following features:")
#print(tsne_data.dtypes)

# Takes a 2D data embedding and an svm trained on it and plots the decision
def plotFrontier(embedded, svm, technique_name, scale):
    # get all the points in the space, and query the svm on them
    xx, yy = numpy.meshgrid(numpy.linspace(min(embedded[:,0])*scale,
                                              max(embedded[:,0])*scale, 500),
                            numpy.linspace(min(embedded[:,1])*scale,
                                              max(embedded[:,1])*scale, 500))
    Z = svm.decision_function(numpy.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape) # positive Z means yes. negative means outliers.

    pyplot.figure(figsize=(20,20))
    pyplot.title('Decision boundary of One-class SVM in '+technique_name+' s
    pyplot.contourf(xx, yy, Z, levels=numpy.linspace(Z.min(), 0, 7), cmap=py
    pyplot.contour(xx, yy, Z, levels=[0], linewidths=2, colors='green') # th
    pyplot.contourf(xx, yy, Z, levels=[0, Z.max()], colors='lightgreen')

    pyplot.scatter(embedded[:, 0], embedded[:, 1], s=10, c='grey')
    for i,song in data.iterrows():
        if random() < show_percent*0.005: # randomly label % of points
            #if song['Artist Name(s)'] in ['Coldplay']:
                x, y = embedded[i]
                pyplot.annotate(song['Track Name'], (x,y), size=10,
                                xytext=(-30,30), textcoords='offset points',
                                ha='center',va='bottom',
                                arrowprops={'arrowstyle':'->', 'color':'red'})

tsne_embedded = TSNE(n_components=2).fit_transform(numerical_data)
```

```
svm_tsne = OneClassSVM(gamma='scale')  
svm_tsne.fit(tsne_embedded)
```

```
plotFrontier(tsne_embedded, svm_tsne, 't-SNE', 1.2)
```

```
using: ['Release Date', 'Duration (ms)', 'Popularity', 'Danceability', 'Energy', 'Key', 'Loudness', 'Mode', 'Speechiness', 'Acousticness', 'Instrumentalness', 'Liveness', 'Valence', 'Tempo', 'Time Signature']
```

```

ValueError                                Traceback (most recent call last)
Cell In[64], line 50
     44         x, y = embedded[i]
     45         pyplot.annotate(song['Track Name'], (x,y), size=10,
     46                        xytext=(-30,30), textcoords='offset points',
     47                        ha='center',va='bottom',
     48                        arrowprops={'arrowstyle':'->', 'color':'red'})
--> 50 tsne_embedded = TSNE(n_components=2).fit_transform(numerical_data)
     52 svm_tsne = OneClassSVM(gamma='scale')
     53 svm_tsne.fit(tsne_embedded)

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/utils/_set_output.py:316, in _wrap_method_output.<locals>.wrapped(self, X, *args, **kwargs)
     314 @wraps(f)
     315 def wrapped(self, X, *args, **kwargs):
--> 316     data_to_wrap = f(self, X, *args, **kwargs)
     317     if isinstance(data_to_wrap, tuple):
     318         # only wrap the first output for cross decomposition
     319         return_tuple = (
     320             _wrap_data_with_container(method, data_to_wrap[0], X, self),
     321             *data_to_wrap[1:],
     322         )

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/base.py:1365, in _fit_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
     1358     estimator._validate_params()
     1360 with config_context(
     1361     skip_parameter_validation=(
     1362         prefer_skip_nested_validation or global_skip_validation
     1363     )
     1364 ):
-> 1365     return fit_method(estimator, *args, **kwargs)

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/manifold/_t_sne.py:1144, in TSNE.fit_transform(self, X, y)
     1123 """Fit X into an embedded space and return that transformed output.
     1124
     1125 Parameters
     1126 (...)
     1141     Embedding of the training data in low-dimensional space.
     1142 """
     1143 self._check_params_vs_input(X)
-> 1144 embedding = self._fit(X)
     1145 self.embedding_ = embedding
     1146 return self.embedding_

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/manifold/_t_sne.py:870, in TSNE._fit(self, X, skip_num_points)
     867     self.learning_rate_ = self.learning_rate
     869 if self.method == "barnes_hut":
-> 870     X = validate_data(
     871         self,

```

```

872         X,
873         accept_sparse=["csr"],
874         ensure_min_samples=2,
875         dtype=[np.float32, np.float64],
876     )
877 else:
878     X = validate_data(
879         self,
880         X,
881         accept_sparse=["csr", "csc", "coo"],
882         dtype=[np.float32, np.float64],
883     )

```

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/utils/validation.py:2954, in validate_data(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)

```

2952         out = X, y
2953 elif not no_val_X and no_val_y:
-> 2954     out = check_array(X, input_name="X", **check_params)
2955 elif no_val_X and not no_val_y:
2956     out = _check_y(y, **check_params)

```

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/utils/validation.py:1105, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, force_all_finite, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)

```

1099     raise ValueError(
1100         f"Found array with dim {array.ndim},"
1101         f" while dim <= 2 is required{context}."
1102     )
1104 if ensure_all_finite:
-> 1105     _assert_all_finite(
1106         array,
1107         input_name=input_name,
1108         estimator_name=estimator_name,
1109         allow_nan=ensure_all_finite == "allow-nan",
1110     )
1112 if copy:
1113     if _is_numpy_namespace(xp):
1114         # only make a copy if `array` and `array_orig` may share mem
ory`

```

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/utils/validation.py:120, in _assert_all_finite(X, allow_nan, msg_dtype, estimator_name, input_name)

```

117 if first_pass_isfinite:
118     return
--> 120 _assert_all_finite_element_wise(
121     X,
122     xp=xp,
123     allow_nan=allow_nan,
124     msg_dtype=msg_dtype,
125     estimator_name=estimator_name,
126     input_name=input_name,
127 )

```

```

File /srv/conda/envs/notebook/lib/python3.10/site-packages/sklearn/utils/validation.py:169, in _assert_all_finite_element_wise(X, xp, allow_nan, msg_dtype, estimator_name, input_name)
    152 if estimator_name and input_name == "X" and has_nan_error:
    153     # Improve the error message on how to handle missing values in
    154     # scikit-learn.
    155     msg_err += (
    156         f"\n{estimator_name} does not accept missing values"
    157         " encoded as NaN natively. For supervised learning, you might want"
    158         "(...)
    159         "#estimators-that-handle-nan-values"
    160     )
--> 169 raise ValueError(msg_err)

```

ValueError: Input X contains NaN.

TSNE does not accept missing values encoded as NaN natively. For supervised learning, you might want to consider `sklearn.ensemble.HistGradientBoostingClassifier` and `Regressor` which accept missing values encoded as NaNs natively. Alternatively, it is possible to preprocess the data, for instance by using an imputer transformer in a pipeline or drop samples with missing values. See <https://scikit-learn.org/stable/modules/impute.html> You can find a list of all estimators that handle NaN values at the following page: <https://scikit-learn.org/stable/modules/impute.html#estimators-that-handle-nan-values>

The point scatter looks really different every time this runs, because it's stochastic. The clusters don't necessarily have sensible interpretations, though you might be able to label a few of them. It's good to see some notionally similar pieces ending up near each other. You can try this with a subset of these dimensions to try to make the result more interpretable.

Modifying the parameters of the SVM changes its fit significantly, so I'm not sure this is the best model. Gamma too large just clearly overfits the data. Gamma too small just makes the decision boundary a boring ellipse. Using `gamma='scale'` as [the docs](#) recommend is a more interesting middle ground, but still the SVM seems to believe that a great many of the songs I love fall outside the boundary.

I'll try a different dimensionality reduction technique. [The original author](#) uses Principle Component Analysis to feed the SVM.

```

In [ ]: from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_embedded = pca.fit_transform(numerical_data)
print("% variance explained by successive PCA dimensions:",
      pca.explained_variance_ratio_)

svm_pca = OneClassSVM(gamma='scale')
svm_pca.fit(pca_embedded)

plotFrontier(pca_embedded, svm_pca, 'PCA', 1)

```

Ideally, songs falling nearer the center here, like Cheeseburger in Paradise and RAC's We Belong, are those that most characterize my taste numerically, and the odd ones, like Pink Floyd's Comfortably Numb and The Fellowship of the Ring orchestral suite, fall on the outside.

So in the end my music taste is a blob that doesn't even fit the data very well. And that's the point: Like many things, it's too complicated to boil down. You can't answer the question fully. But understanding elements of the answer can aid the process of discovery, and that's valuable. It's why Spotify is such a force at music recommendation. It's why Data Science.